

()AsEnumerable

من "IQueryable" إلى "IEnumerable" بدون تنفيذ ال Query

1. ايه هو "AsEnumerable"؟

لما تشوف الكود ده:

```
.AsEnumerable()
```

ممکن جدًا تحس إن ال method دي ملهاش لازمة.

خصوصًا إن ال implementation بتاعها بسيط جدًا:

```
public static IEnumerable<T> AsEnumerable<T>(  
    this IEnumerable<T> source)  
{  
    return source;  
}
```

وده فعلاً يخلينا نسأل:

هل هي بترجع نفس ال object؟

هل بتعمل collection جديدة؟

هل بتنفيذ ال query؟

لو مفيش حاجة من دول بتحصل، إيه لازمتها؟

الإجابة المهمة هنا:

اهمية ال "AsEnumerable" مش في إنها تغيّر البيانات، ولكن في إنها تغيّر نوع ال sequence
اللي ال compiler بيتعامل معاه.

والتغيير ده بيأثر على ال LINQ operators اللي بتيجي بعدها.

2. المشكلة الأساسية: "IQueryable" مقابل "IEnumerable"

خلينا نفترض إن عندنا:

```
IQueryable<Customer> customers = dbContext.Customers;
```

ولما نكتب:

```
customers.Where(c => c.Id > 10);
```

ال compiler هيختار ال "Where" الموجودة في:

"Queryable"

وده لأن ال source نوعه:

"IQueryable"

وبالتالي EF Core يقدر يتعامل مع ال query كـ "Expression Tree" ويحاول يترجمها إلى SQL.

ال flow هنا بيكون تقريبًا:

```
IQueryable
↓
Queryable
↓
Expression Tree
↓
EF Core
↓
SQL
↓
Database
```

3. طب ايه اللي بيحصل لما نستخدم "AsEnumerable"؟

لو كتبنا:

```
customers
    .AsEnumerable()
    .Where(c => CustomMethod(c));
```

هنا حصل تغيير مهم.

قبل الـ "AsEnumerable" :

"IQueryable"

بعد الـ "AsEnumerable" :

"IEnumerable"

وبالتالي الـ "Where" اللي بعدها أصبحت من:

"Enumerable"

بدل:

"Queryable"

وده معناه إننا انتقلنا من تنفيذ ال query عن طريق ال provider إلى:

"LINQ to Objects"

بمعنى أبسط:

الـ "AsEnumerable" بتحدد إن الـ LINQ operators اللي بعدها تشتغل على الـ objects الموجودة في الـ application بدل ما تترجم كجزء من الـ database query.

4. ال Pipeline بالكامل

قبل الـ "AsEnumerable" :

```
IQueryable<T>
  ↓
Queryable.*
  ↓
Expression Tree
  ↓
EF Core
  ↓
SQL
  ↓
Database
```

بعد الـ "AsEnumerable" :

```
IEnumerable<T>
  ↓
Enumerable.*
  ↓
LINQ to Objects
  ↓
.NET
  ↓
C# Execution
```

ومن هنا نستنتج التالي :

"Queryable" معناها إن ال query لسه تحت سيطرة ال provider، زي EF Core.
"Enumerable" معناها إن LINQ هيشغل على objects موجودة بالفعل في ال application.
"AsEnumerable" هي نقطة الانتقال بين المرحلتين.

5. تعالى نشوف مثال كامل

شوف ال query دي:

```
var query = dbContext.Customers
    .Where(c => c.Id > 10)
    .AsEnumerable()
    .Where(c => CustomMethod(c));
```

هنا عندنا "Where" مرتين، لكن الاتنين مش بيعملوا نفس الحاجة.

ال "Where" الأولى

```
.Where(c => c.Id > 10)
```

ال "Where" دي مرتبطة بـ:

"Queryable"

لأننا لسه بنتعامل مع:

"IQueryable"

وبالتالي EF Core يقدر يترجم الشرط إلى SQL.

تقريبًا:

```
SELECT *  
FROM Customers  
WHERE Id > 10
```

بعد "AsEnumerable"

بعد:

```
.AsEnumerable()
```

ال sequence أصبحت تُعامل كـ:

"IEnumerable"

وده معناه إن ال "Where" اللي بعدها هتكون من:

"Enumerable"

وبالتالي:

```
.Where(c => CustomMethod(c))
```

هتتنفذ في ال application باستخدام #C.

ال flow يصبغ:

```
Database
  ↓
WHERE Id > 10
  ↓
Filtered Rows
  ↓
Application Memory
  ↓
CustomMethod()
  ↓
Final Result
```

6.ليه محتاجين "IEnumerable"؟

أحيانًا يكون عندك query فيها جزء EF Core يقدر يترجمه إلى SQL، وجزء تاني مش قادر يترجمه.

مثال:

```
Regex regex = new Regex(...);
```

ولو حاولت تستخدم:

```
regex.Matches(...)
```

داخل ال query:

```
var articles = dbContext.Articles
    .Where(a => regex.Matches(a.Text).Count < 100);
```

أنت هنا بتطلب من EF Core إنه يحوّل:

```
regex.Matches(a.Text)
```

إلى SQL.

ولو مفيش translation مناسبة للعملية دي، ال query هتفشل.

#7. الحل: تقسيم ال Query

ممکن نخلي الجزء اللي EF Core يفهمه يفضل في ال database:

```
var articles = dbContext.Articles
    .Where(a => a.Topic == "influenza")
    .AsEnumerable()
    .Where(a => regex.Matches(a.Text).Count < 100);
```

دلوقتي عندنا مرحلتين.

أولاً: Database Side

الجزء ده:

```
.Where(a => a.Topic == "influenza")
```

EF Core يقدر يترجمه إلى SQL.

تقريبًا:

```
SELECT *  
FROM Articles  
WHERE Topic = 'influenza'
```

ثانيًا: Application Side

بعد:

```
.AsEnumerable()
```

الـ "Where" التالية أصبحت من:

"Enumerable"

وبالتالي:

```
.Where(a => regex.Matches(a.Text).Count < 100)
```

تتنفذ في C#.

فال execution سيكون:

```
Database
  ↓
Filter by Topic
  ↓
Matching Articles
  ↓
Application Memory
  ↓
Regex.Matches()
  ↓
Final Result
```

وده بيسمح لنا إننا نستخدم ال database في الجزء اللي تقدر تنفذه بكفاءة، وننقل فقط الجزء اللي محتاج #C لل application.

8. أهم نقطة: "AsEnumerable" مش معناها Execution

دي من أهم النقاط اللي لازم تثبت.

لما تعمل:

```
var query = dbContext.Customers
    .Where(c => c.Id > 10)
    .AsEnumerable();
```

ال query لسه ما اتنفذتش لمجرد إنك استدعيت:

```
.AsEnumerable()
```

ليه؟

لأن "AsEnumerable" لا تقوم بعمل enumeration.

هي فقط بتخلي ال sequence يتم التعامل معاها على إنها:

"IEnumerable"

وال execution يحصل لما تبدأ فعلاً في استهلاك ال sequence.

مثلاً:

```
foreach (var customer in query)
{
    // ...
}
```

أو:

```
var list = query.ToList();
```

9. "AsEnumerable" مقابل "ToList"

"AsEnumerable"

```
var query = dbContext.Customers
    .Where(c => c.Id > 10)
    .AsEnumerable();
```

خصائصها:

مش بتنقذ ال query على طول.

مش بتعمل database call أول ما تستدعيها.

مش بتعمل collection جديدة.

مش بتجيب البيانات وتحطها في ال memory.

بترجع `"IEnumerable<T"` .

بتخلّي ال LINQ operators اللي بعدها تشتغل باستخدام `"Enumerable"` بدل `"Queryable"` .

بتسمحلك تنقل جزء من ال query عشان يتنفّذ جوّه ال `#C` بدل ال `database`.

"ToList"

```
var customers = dbContext.Customers
    .Where(c => c.Id > 10)
    .ToList();
```

خصائصها:

بتنفّذ ال query على طول.

بتعمل `database call`.

بتجيب النتائج من ال `database`.

بتحوّل النتائج لبيانات موجودة فعلاً في ال `memory`.

بتخزّن النتائج في ال `memory`.

بترجع `"List<T"` .

10. الفرق في جدول واحد

النقطة	"ASENUMERABLE"	"TOLIST"
تنفيذ ال Query فورًا	لا	نعم
Database Call فورًا	لا	نعم
إنشاء Collection جديدة	لا	نعم
Materialization	لا	نعم
نوع النتيجة	"IEnumerable"	"List"
التحول إلى "Enumerable"	نعم	نعم

الفرق الأساسي:

الـ `"AsEnumerable"` معناها: **غير طريقة التعامل مع باقي ال LINQ pipeline**.

بينما:

الـ `"ToList"` معناها: **نُفذ ال query الآن وحزّن النتائج في List**.

11. خذ بالك من "AsEnumerable"

المشكلة مش في استخدام `"AsEnumerable"`.

المشكلة في إنك تستخدمها في المكان الغلط.

مثلاً:

```
var articles = dbContext.Articles
    .AsEnumerable()
    .Where(a => a.Topic == "influenza");
```

هنا أنت نقلت ال filtering نفسه إلى ال application.

يعني بدل ما يحصل:

```
Database
  ↓
WHERE Topic = 'influenza'
  ↓
Filtered Rows
  ↓
Application
```

ممکن يحصل:

```
Database
  ↓
Load Articles
  ↓
Application Memory
  ↓
Where(...)
```

ولو عندك مليون record، أنت ممكن تحمل كمية ضخمة من البيانات لل application ثم تعمل filtering هناك.

وده طبعا مش المطلوب.

12. المكان الأفضل

بدل:

```
dbContext.Articles
    .AsEnumerable()
    .Where(a => a.Topic == "influenza");
```

استخدم:

```
dbContext.Articles
    .Where(a => a.Topic == "influenza")
    .AsEnumerable();
```

كده ال filtering يحصل في ال database الأول.

وبعدها فقط تنتقل لل application:

```
Database
  ↓
WHERE Topic = 'influenza'
  ↓
Filtered Results
  ↓
Application
```

والقاعدة العملية هنا:

حاول تخلي أكبر قدر ممكن من ال *filtering* وال *processing* القابل للترجمة يحصل في ال database قبل `"AsEnumerable"`.

13. لماذا ال Implementation تبدو عديمة الفائدة؟

نرجع لل implementation:

```
public static IEnumerable<T> AsEnumerable<T>(  
    this IEnumerable<T> source)  
{  
    return source;  
}
```

هي فعلاً لا تعمل transformation على البيانات.

لكن ده مش الهدف منها.

الهدف هو تغيير النوع الذي يتعامل معه ال compiler في باقي ال expression.

قبل "AsEnumerable":

```
IQueryable<T>  
↓  
Queryable.Where()
```

بعد "AsEnumerable":

```
IEnumerable<T>  
↓  
Enumerable.Where()
```

فال object نفسه ممكن يفضل زي ما هو.

لكن ال type اللي ال compiler شايفه اتغير.

وده كفاية جدًا عشان يغير ال LINQ operator اللي هيتم اختياره.

14. ال Mental Model

أفضل طريقة تحفظ بيها "AsEnumerable" إنك تعتبرها:

Boundary بين ال Database Query وال Local LINQ Execution.

AsEnumerable()



Database / Provider Side

IQueryable<T>



Queryable.*



Expression Tree



EF Core



SQL



Database

Application Side

IEnumerable<T>



Enumerable.*



LINQ to Objects



C# / .NET



Local Execution

Final Takeaway

لما تشوف:

```
.AsEnumerable()
```

ما تفكرش:

"هاتلي البيانات".

فكر:

"من هنا ورا، خلي الـ LINQ يشتغل كـ LINQ to Objects".

بالتالي:

```
Before AsEnumerable
    ↓
IQueryable
    ↓
Queryable
    ↓
Provider
    ↓
SQL
```

وبعدھا:

```
AsEnumerable
  ↓
IEnumerable
  ↓
Enumerable
  ↓
LINQ to Objects
  ↓
C#
```

وأهم جملة في التوبيك كله:

"AsEnumerable" لا تنفذ ال query؛ هي فقط تغيّر ال LINQ pipeline بحيث إن ال operators اللي بعدها يتم تنفيذها كـ LINQ-to-Objects.

The Curious Backend Developer